

API 文件前置作業

目錄

- 1.1 API 認證
- 1.2 狀態碼
- 1.3 轉真人 Webhook 事件

API 認證

MaiAgent 使用 API 金鑰進行身分驗證。

身分認證方式

所有 API 請求必須在 HTTP Header 中包含 API Key，格式如下：

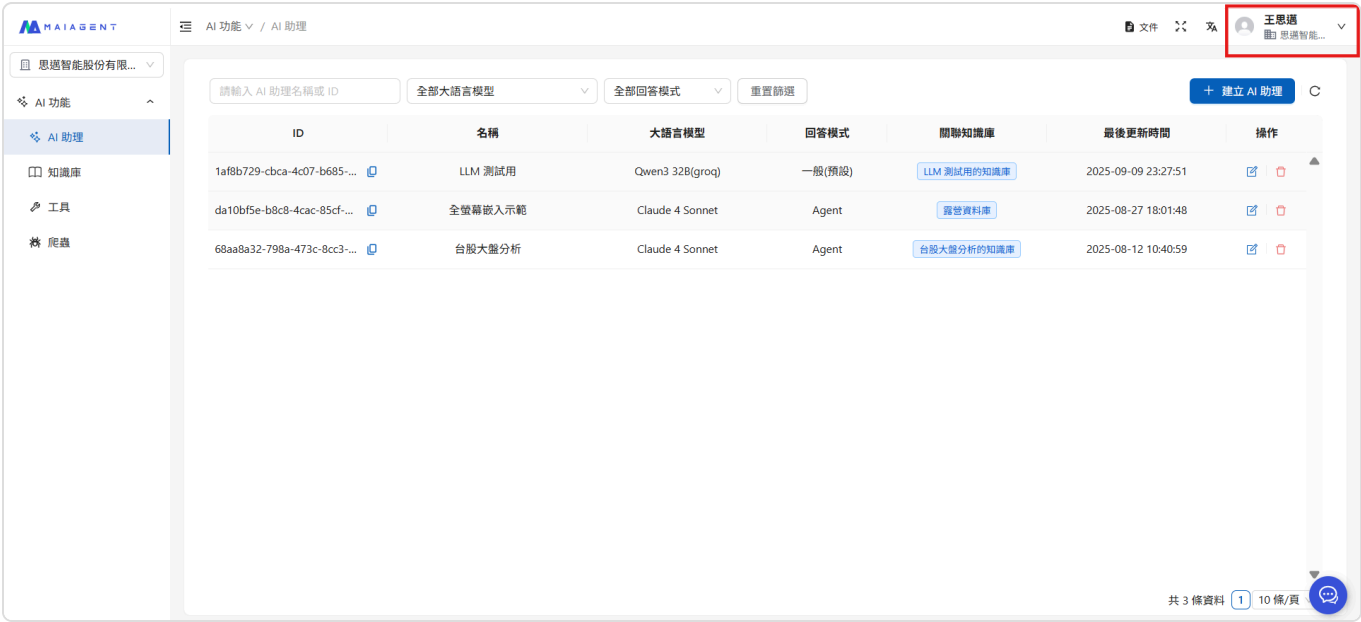
```
const headers = {
  "Authorization": "Api-Key YOUR_API_KEY_HERE",
  "Content-Type": "application/json"
}
```

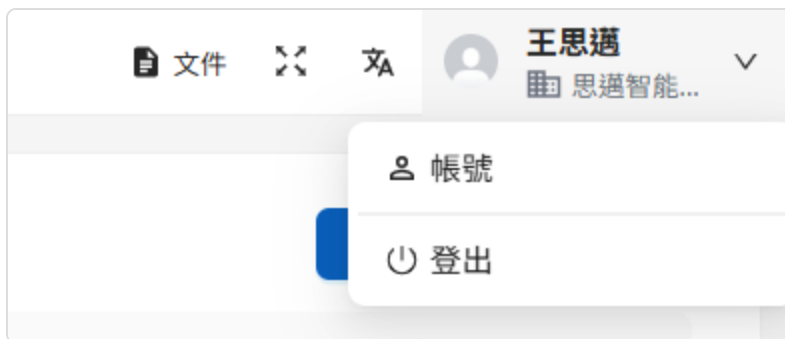
如何獲取 API 金鑰？

請登入 MaiAgent 系統後台，然後按照以下步驟操作：

點擊右上角的「使用者名稱」下拉選單

點擊「帳號」進入個人資料頁面





將頁面切換至 API 金鑰頁面。



即可複製查看「API 金鑰」。



安全提醒：請妥善保管您的 API 金鑰，不要在公開場所分享 金鑰格式說明：API 金鑰英數字組合

錯誤處理

當認證錯誤時，API 會回傳 401 Unauthorized 的 HTTP 錯誤代碼：

API Key 無效或缺失

```
{
  "detail": "Authentication credentials were not provided."
}
```

修復建議：檢查請求標頭是否包含正確的 *Authorization* 欄位。格式應為：`Authorization: Api-Key YOUR_API_KEY`

API Key 格式錯誤

```
{
  "detail": "Invalid API key format."
}
```

修復建議：確認 API Key 格式正確。檢查是否使用了正確的前綴 `Api-Key` 而非 `Bearer`，並確認 API Key 本身沒有多餘的空格或字元。

狀態碼

在透過 API 送出 HTTP 請求時，可使用以下狀態碼查看 API 請求成功與否：

狀態碼說明

正確代碼

狀態碼	說明
200	請求成功
201	資源成功建立

正確回應範例

此處為內容示範，不同端點的回傳格式內容可能有差異，請以該端點的說明文件參考為主

```
{
  "count": "integer",
  "next": "string|null",
  "previous": "string|null",
  "results": [
    // 結果回傳內容
  ]
}
```

錯誤代碼

狀態碼	說明
400	請求格式錯誤
401	認證失敗
403	權限不足
404	資源不存在
500	伺服器錯誤

錯誤回應範例

400 - 請求錯誤：必填欄位缺失

```
{
  "files": ["This field is required."],
  "filename": ["This field is required."],
  "file": ["This field is required."]
}
```

轉真人 Webhook 事件

當客服入口（收件匣）完成「轉真人」指派時，MaiAgent 會以 HTTPS POST 將一筆 `handoff.created` 事件推送到你設定的端點，讓派工、告警或 CRM 系統自動接手，不需輪詢；對話回到 AI 時，同一個端點會收到 `handoff.returned_to_ai`，讓接手中的系統知道轉真人已結束。

啟用方式

由客戶的管理者或串接人員在後台自助完成，毋需工程協助：

後台 → 設定 → 收件匣 → 選擇收件匣 → 「轉真人」頁籤 → 「通知」分頁
開啟「Webhook 通知（系統串接）」開關，填入接收端網址（僅接受 `https://` 開頭）
（選填但建議）填入簽章密鑰，啟用來源驗證（見下方「簽章驗證」）

Webhook 與通知中心、Email 是三個獨立開關，互不影響；預設關閉、逐收件匣啟用。此通道與收件匣既有的「訊息 Webhook」（訊息收送）互不相干。

事件種類

event	何時送出	備註
<code>handoff.created</code>	對話被指派給真人客服（含手動指派、自行認領、轉接、自動指派）	<code>assigneeId</code> 為被指派客服；轉接給另一位客服會再送一筆
<code>handoff.returned_to_ai</code>	客服在後台按「轉回 AI Agent」或透過 <code>return-to-ai</code> API 將對話交回 AI	<code>assigneeId</code> 為 <code>null</code> （對話已無負責客服）

兩種事件的 payload 鍵集、簽章與重試行為完全相同，接收端依 `event` 欄位分流；未來可能新增事件種類，遇到不認識的 `event` 請回 2xx 並忽略，不要當成錯誤。

`handoff.returned_to_ai` 的語意是「專員主動把對話轉回 AI」，不等於「轉真人流程結束」：客服直接把對話標為「已解決」、或客服因失去收件匣權限而使對話重新排隊時，不會送出此事件。需要完整的結案訊號時，請以對話 API 查詢 `status`，或在訊息 Webhook 的 `conversation.progress_status` 觀察狀態變化。

事件格式

```
POST <你設定的網址>
Content-Type: application/json
```

```
{
  "event": "handoff.created",
  "conversationId": "6c2b7c1e- ... ",
  "inboxId": "0f9a3d42- ... ",
  "organizationId": "b1d20c77- ... ",
  "assigneeId": "3e8f5a90- ... ",
  "contactId": "a4c1f6b3- ... ",
  "conversationUrl": "https://admin.maiagent.ai/ ... "
}
```

欄位	類型	說明
event	string	<code>handoff.created</code> 或 <code>handoff.returned_to_ai</code> （見上方「事件種類」）
conversationId	string (uuid)	對話識別碼
inboxId	string (uuid)	收件匣識別碼
organizationId	string (uuid)	組織識別碼
assigneeId	string (uuid)	null
contactId	string (uuid)	null
conversationUrl	string (uri)	後台對話頁連結

Payload 只含識別資訊與後台連結，**不含對話文字內容**；需要對話內容時，請以識別碼呼叫對話與訊息 API 查詢。

簽章驗證（建議啟用）

在後台設定簽章密鑰後，每次推送會帶下列標頭：

```
X-MaiAgent-Signature-256: sha256=
```

值為以密鑰對 **原始 request body bytes** 計算的 HMAC-SHA256 十六進位摘要。驗證時務必使用收到的原始 bytes，不要先解析再重新序列化（欄位順序與空白差異會使摘要不一致）。


```
import hashlib
import hmac

def verify_signature(raw_body: bytes, header_value: str, secret: str) → bool:
    expected = 'sha256=' + hmac.new(secret.encode(), raw_body,
hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, header_value)
```

密鑰為 write-only：儲存後不回顯，後台僅顯示「已設定」；欄位留空儲存代表維持既有密鑰，更換直接輸入新值，移除請使用「清除密鑰」操作。未設定密鑰時，推送不帶簽章標頭。

接收端最小範例

以下是一支可直接啟動的 Python (Flask) 接收端：讀取原始 bytes 驗章、依 `event` 分流、先回 2xx 再做耗時處理。把 `HANDOFF_WEBHOOK_SECRET` 換成後台設定的簽章密鑰即可。

```

import hashlib
import hmac
import os

from flask import Flask, abort, request

app = Flask(__name__)
SECRET = os.environ["HANDOFF_WEBHOOK_SECRET"]

def verify_signature(raw_body: bytes, header_value: str | None) → bool:
    if not header_value:
        return False
    expected = "sha256=" + hmac.new(SECRET.encode(), raw_body,
hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, header_value)

@app.post("/webhooks/handoff")
def handoff_webhook():
    raw = request.get_data() # 原始 bytes，不要用 request.json 再序列化
    if not verify_signature(raw, request.headers.get("X-MaiAgent-Signature-
256")):
        abort(401)

    event = request.get_json(force=True)
    key = (event["event"], event["conversationId"]) # 重試會送完全相同的內容，用這組去
重

    if event["event"] == "handoff.created":
        # 在你的系統開單／建立會話；轉接給另一位客服時會再收到一筆，assigneeId 會變
        open_ticket(event["conversationId"], event["assigneeId"],
event["conversationUrl"])
    elif event["event"] == "handoff.returned_to_ai":
        close_ticket(event["conversationId"])
    # 未知事件：回 2xx 忽略，不要當錯誤

    return "", 204 # 10 秒內回 2xx；耗時工作丟到背景

def open_ticket(conversation_id, assignee_id, url):
    ...

def close_ticket(conversation_id):
    ...

```

```
HANDOFF_WEBHOOK_SECRET=你的密鑰 flask --app receiver run --port 8080
```

本機測試可用下列指令模擬一筆推送（簽章用同一把密鑰算）：

```
BODY='{ "event": "handoff.created", "conversationId": "6c2b7c1e-0000-0000-0000-000000000001", "inboxId": "0f9a3d42-0000-0000-0000-000000000002", "organizationId": "b1d20c77-0000-0000-0000-000000000003", "assigneeId": "3e8f5a90-0000-0000-0000-000000000004", "contactId": "a4c1f6b3-0000-0000-0000-000000000005", "conversationUrl": "https://admin.maiagent.ai/conversations/index?conversationId=6c2b7c1e-0000-0000-0000-000000000001"}'
SIG=$(printf '%s' "$BODY" | openssl dgst -sha256 -hmac "你的密鑰" | awk '{print $NF}')
curl -X POST http://localhost:8080/webhooks/handoff
  -H "Content-Type: application/json"
  -H "X-MaiAgent-Signature-256: sha256=$SIG"
  -d "$BODY"
```

回應與重試

接收端應在 10 秒內以 2xx 狀態碼回應，回應內容不拘。

逾時、連線失敗或非 2xx 回應時，系統會自動重試，最多 3 次、間隔 60 秒；重試送出的內容與原始推送完全相同，接收端請自行幂等處理（例如以 `event` + `conversationId` 去重）。

Webhook 推送失敗不影響轉真人主流程，也不影響通知中心與 Email 通知。

轉真人後由外部系統接手對話

若你的真人客服不在 MaiAgent 後台操作，而是在自己的客服系統回覆，可以把本頁的事件與收件匣既有的「訊息 Webhook」、訊息 API 組合起來，讓對話在轉真人期間雙向同步。所有步驟都用既有能力，不需要額外開通。

串接流程

得知轉真人開始：收到 `handoff.created` 後，在你的系統以 `conversationId` 開單或建立會話，並記下 `assigneeId`、`contactId`。

接收客戶訊息與客服回覆：在後台該收件匣的「訊息 Webhook」新增 `incoming`（客戶訊息）與 `outgoing`（AI 或客服回覆）方向的 Webhook。每筆訊息的 `conversation` 物件帶有目前處理狀態，只處理 `progress_status` 為 `human_serving` 的訊息即可略過 AI 模式的流量：

欄位	說明
conversation.status	open queued resolved
conversation.progress_status	ai_processing waiting_for_human human_serving resolved open
conversation.auto_reply_enabled	AI 是否會自動回覆；轉真人後為 false
conversation.assignee	負責客服 { "id", "name" } ，無人負責時為 null
sender.type	contact (客戶) user (真人客服) chatbot (AI)

把客服回覆寫回對話：以 API 金鑰呼叫 `POST /api/v1/messages/outgoing/`，帶 `conversation` 與 `content`，MaiAgent 會把訊息轉發到客戶所在的渠道。訊息的發送者為該 API 金鑰所屬的成員，該成員需具備此收件匣的存取權限（詳見「對話與訊息」端點說明）。

結束轉真人：客服處理完畢後呼叫 `PATCH /api/v1/conversations/{id}/return-to-ai/`，AI 重新接手；若客服是在 MaiAgent 後台按「轉回 AI Agent」，你會收到 `handoff.returned_to_ai`，請據以關閉你系統中的會話。

呼叫範例

（1）轉真人期間收到的訊息 Webhook（`maiaagent_standard` 格式，節錄）—— 用 `conversation.progress_status` 判斷是否已在真人模式、`sender.type` 判斷發話者：

```
{
  "id": "9d2f7a10- ... ",
  "type": "incoming",
  "content": "我要改訂單地址",
  "created_at": "2026-09-09T02:30:00+00:00",
  "conversation": {
    "id": "6c2b7c1e- ... ",
    "title": " ... ",
    "type": "individual",
    "status": "open",
    "progress_status": "human_serving",
    "auto_reply_enabled": false,
    "assignee": { "id": "3e8f5a90- ... ", "name": "客服小美" }
  },
  "inbox": { "id": "0f9a3d42- ... ", "name": "官網客服", "channel_type": "web" },
  "contact": { "id": "a4c1f6b3- ... ", "name": "王小明", "email": null,
"phone_number": null },
  "sender": { "id": "a4c1f6b3- ... ", "name": "王小明", "type": "contact" },
  "attachments": [],
  "metadata": {}
}
```

(2) 把客服回覆寫回對話—— `conversation` 與 `content` 即可，其餘欄位選填；訊息會以 API 金鑰所屬成員的身分送到客戶所在渠道：

```
curl -X POST "https://api.maiagent.ai/api/v1/messages/outgoing/"
-H "Authorization: Api-Key YOUR_API_KEY"
-H "Content-Type: application/json"
-d '{
  "conversation": "6c2b7c1e- ... ",
  "content": "您好，地址已為您更新為台北市信義區..."
}'
```

(3) 處理完畢，把對話交回 AI——不需要 request body，成功回 200 與對話物件；之後 AI 恢復自動回覆、`assignee` 清空：

```
curl -X PATCH "https://api.maiagent.ai/api/v1/conversations/6c2b7c1e- ... /return-to-ai/"
-H "Authorization: Api-Key YOUR_API_KEY"
```

三步合在一起（Python，`requests`）：

```

import requests

API = "https://api.maiagent.ai/api/v1"
HEADERS = {"Authorization": "Api-Key YOUR_API_KEY"}

def on_message_webhook(payload: dict) → None:
    conv = payload["conversation"]
    if conv["progress_status"] ≠ "human_serving":
        return # AI 模式的流量不進客服系統
    if payload["sender"]["type"] = "contact":
        push_to_agent_console(conv["id"], payload["content"]) # 客戶訊息→你的客服介
面

def reply(conversation_id: str, text: str) → None:
    requests.post(f"{API}/messages/outgoing/", headers=HEADERS,
                  json={"conversation": conversation_id, "content": text},
                  timeout=10).raise_for_status()

def finish(conversation_id: str) → None:
    requests.patch(f"{API}/conversations/{conversation_id}/return-to-ai/",
                  headers=HEADERS, timeout=10).raise_for_status()

```

注意事項

透過 API 寫回的回覆會再經由 `outgoing` 訊息 Webhook 外送一次（`sender.type` 為 `user`）；請以訊息 `id` 去重，避免在你的系統重複顯示。

`handoff.created` 在轉接給另一位客服時會再送一筆，`assigneeId` 為新的負責人；請以 `conversationId` 更新既有會話而非重複開單。

訊息 Webhook 與轉真人 Webhook 是兩組獨立設定與簽章方式：前者以 `X-Webhook-Secret` 標頭原樣帶出設定的密鑰，後者為本頁的 HMAC-SHA256 簽章。